Pandas II: CRUD & Query over Dataframes

Wook-Shin Han

Spring 2026

Learning Objectives

- **CRUD for pandas**: Create, Update, Delete, Query.
- **ERA Operators**: Formal definitions + pandas counterparts.
- **Grouping/Pivoting**: ERA γ (grouping/aggregation) and pivot tables.
- **Semantics**: State the ERA query and describe its meaning before code.

Toy Dataset

```
import pandas as pd
import numpy as np

data = {'State': ['CA']*4, 'Sex': ['F']*4, 'Year': [1910]*4,
        'Name': ['Mary', 'Helen', 'Dorothy', 'Margaret'],
        'Count': [295, 239, 220, 163]}
babynames = pd.DataFrame(data)
print(babynames.shape)
```

CRUD in pandas — API → Behavior

Op	API	What it does (ERA mapping)
Create	`pd.concat([df1,df2], axis=0)`	Stack rows (bag append). For set semantics, add `.drop_duplicates()`. (ERA: $\cup + \delta$)
Create	`pd.concat([df1,df2], axis=1)`	Bind columns by index alignment. (ERA: reshape / attribute extension)
Create	`df.assign(new=expr,...)`	Return a new DF with computed/updated columns; original unchanged. (ERA: generalized projection π^*)
Create	`df.loc[new_idx] = row`	Insert/overwrite a row by label (mutation). (Not pure RA)
Create	`df.insert(pos,"col",values)`	Insert a column at a given position (mutation). (ERA: π^*)
Create	`df.reindex(new_index/columns)`	Conform to new index/columns; creates missing rows/cols. (reshape)
Update	`df.loc[keys,cols] = values`	Set by *label*; vectorized and index-aligned. (mutation)
Update	`df.at[row,col] = scalar`	Fast scalar set by labels (single cell). (mutation)
Update	`df.where(cond, other)`	Keep where `cond` True else `other`. (ERA: selection as guard σ)
Update	`df.mask(cond, other)`	Replace where `cond` True (inverse of `where`).
Update	`df.replace(old,new)`	Value remapping (dict/list/regex).
Update	`df.assign(col=f(df))`	Functional update (returns new DF). (ERA: π^*)
Delete	`df.drop(labels, axis=0)`	Remove rows by label; returns new DF unless reassigned.
Delete	`df.drop(labels, axis=1)`	Remove columns by label.
Delete	`df.dropna(subset=[...])`	Remove rows with NA in given columns. (ERA: σ on $\mathrm{is_not_null}$)
Delete	`df.pop("col")`	Remove a column and return it as `Series`.
Delete	`del df["col"]`	Delete a column in place; no return.

Query in pandas — API → Behavior

Op	API	What it does (ERA mapping)
Query	`df.loc[mask]`	Filter rows by boolean mask; optional col pick. (ERA: selection σ)
Query	`df.query("expr")`	SQL-like predicate strings over columns. (ERA: σ)
Query	`pd.merge(L,R,on=...,how=...)`	Joins: inner/left/right/outer/cross; equi-join or theta join. (ERA: $\bowtie_\theta$, $\times$)
Query	`df.groupby(keys).agg(...)`	Group by keys and compute aggregates. (ERA: $\gamma_{G;f(\cdot)}$)
Query	`df.drop_duplicates(subset)`	Remove duplicate rows (globally or by subset). (ERA: duplicate elimination δ)
Query	`df.sort_values(by)`	Order rows by keys (asc/desc). (ERA ext.: sorting τ)
Query	`pd.concat([...], axis=0)`	Union-like row stacking (bag); add δ for set. (ERA: $\cup + \delta$)
Query	`pd.crosstab(idx, cols)`	Contingency table of counts by idx $\times$ cols. (ERA: PIVOT with $\mathrm{count}(*)$)
Query	`df.pivot_table(index,columns,values,aggfunc)`	General pivot with aggregation f. (ERA: PIVOT with f)

Create — Examples

Note: Create mutates the relation instance., an ERA query operator

```
# Row: append one
babynames.loc[len(babynames)] = ["CA","F",2023,"Nova",12]

# Rows: batch append (preferred)
rows = [("CA","M",2023,"Atlas",9), ("CA","F",2024,"Mira",7)]
babynames = pd.concat([babynames,
    pd.DataFrame(rows, columns=babynames.columns)],
    ignore_index=True)

# Columns: derive and insert
babynames["decade"] = (babynames["Year"] // 10) * 10
#  make "state_sex" = "State-Sex" and insert it as the leftmost column (in-place)
babynames.insert(0, "state_sex",
    babynames["State"].str.cat(babynames["Sex"], sep="-"))
```

Update — Examples

Note: Update is also not an ERA query; it modifies tuples/attributes.

```
# Point update
ix = (babynames["Name"]=="Mary") & (babynames["Year"]==1910)
babynames.loc[ix, "Count"] = babynames.loc[ix, "Count"] + 1

# Conditional update
low = babynames["Count"] < 10
babynames.loc[low, "Count"] = 0

# Map / replace
babynames["Sex"] = babynames["Sex"].replace({"F":"Female","M":"
    Male"})

# Where/Mask idiom
high = babynames["Count"] >= 100
babynames["pop_flag"] = np.where(high, "popular", "normal")
```

Delete — Examples

Note: Delete removes tuples/attributes; not an ERA query.

```
# Column removal with return
cnt = babynames.pop("Count")  # returns Series
babynames["Count"] = cnt      # add back (if needed)

# Drop rows by labels
to_remove = babynames.index[:2]
babynames = babynames.drop(index=to_remove)

# Drop rows by condition
old_rare = (babynames["Year"] < 1920) & (babynames["Count"] < 5)
babynames = babynames.drop(index=babynames[old_rare].index)
```

Selection $\sigma_\theta(R)$

Formal: $\sigma_\theta(R) = \{\, t \in R \mid \theta(t) \,\}$. Schema preserved.
ERA (Query): $\sigma_{\text{Year}=1910 \wedge \text{Sex startswith 'F'}}(\text{babynames})$
Meaning: Keep only rows where `Year` is 1910 and `Sex` begins with `'F'`.
pandas: `df.loc[mask]`, `df.query("...")`.

```
mask = (babynames["Year"]==1910) & (babynames["Sex"].str.
    startswith("F"))
sel = babynames.loc[mask]
```

Generalized Projection $\pi_{e_1 \to B_1, \ldots, e_k \to B_k}(R)$

Formal: Compute expressions e_i over attributes of R to form new attributes B_i.
ERA (Query): $\pi_{\text{Year, Name, SUBSTR(Name,1,1)} \to \text{first_letter, LEN(Name)} \to \text{name_len}}(\text{babynames})$
Meaning: Project `Year`, `Name`, and derived columns: the first letter of `Name` and the length of `Name` (no aggregation).
pandas: `df.assign(...)[[...]]`.

```
proj_easy = (babynames
             .assign(first_letter=lambda d: d["Name"].str[0],
                     name_len=lambda d: d["Name"].str.len())
             [["Year", "Name", "first_letter", "name_len"]])
```

Rename $\rho_{A \to A'}(R)$

Formal: Change attribute names; values unchanged.
ERA (Query): $\rho_{\text{Count} \to \text{freq}}(\text{babynames})$
Meaning: Rename attribute `Count` to `freq`.
pandas: `df.rename(columns={"Count":"freq"})`.

```
ren = babynames.rename(columns={"Count":"freq"})
```

Duplicate Elimination $\delta(R)$

Formal: Remove duplicate tuples (set semantics).
ERA (Query): $\delta\left(\pi_{\text{Year,Name}}(\text{babynames})\right)$
Meaning: Unique pairs of `(Year, Name)`.
pandas: `df.drop_duplicates()`.

```
unique_year_name = babynames[["Year","Name"]].drop_duplicates()
```

Sorting $\tau_{A_1^{\uparrow/\downarrow},\ldots}(R)$ (extended)

Formal: Order tuples by keys (not core RA but used in ERA).
ERA (Query): $\tau_{\text{Year}^{\uparrow},\ \text{Count}^{\downarrow}}(\text{babynames})$
Meaning: Sort by `Year` ascending, then `Count` descending.
pandas: `df.sort_values(...)`.

```
sorted_df = babynames.sort_values(["Year","Count"], ascending=[
    True,False])
```

Union $R \cup S$, Intersection $R \cap S$, Difference $R - S$

Formal: Union-compatible relations (same schema).
ERA (Query, set semantics):
$\pi_{\text{Year,Name}}(\text{babynames}) \cup S$, $\pi_{\text{Year,Name}}(\text{babynames}) \cap S$, $\pi_{\text{Year,Name}}(\text{babynames}) - S$
Meaning: Set union/intersection/difference on `(Year, Name)` with a given set S.
pandas (bag→set with `drop_duplicates`):

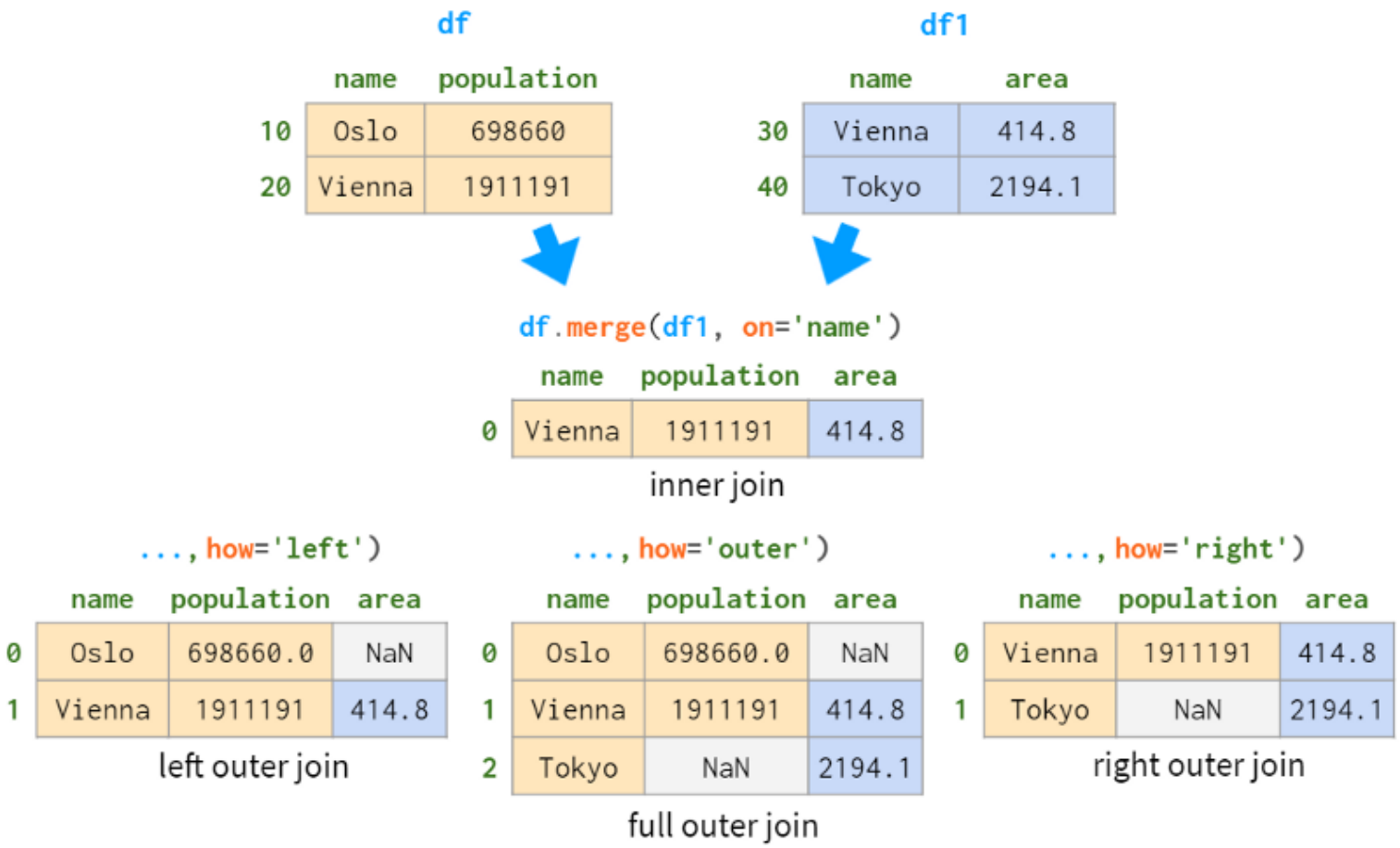
```python
# R: relation with unique (Year, Name) pairs taken from babynames
R = babynames[["Year","Name"]].drop_duplicates()

S = pd.DataFrame({"Year":[1910, 1910], "Name":["Mary", "Alice"]})

# UNION (set union with duplicate elimination)
#   - pd.concat stacks R and S, ignore_index to re-number rows
union_set = pd.concat([R, S], ignore_index=True).drop_duplicates()

# INTERSECTION (set intersection)
#   - inner merge on common columns keeps tuples present in both R and S
inter = R.merge(S, how="inner")

# DIFFERENCE (R - S)
#   - left merge with indicator marks matches vs non-matches
#   - keep only rows that appear only in R
#   - drop the helper column
diff = R.merge(S, how="left", indicator=True)
     .query("_merge == 'left_only'")
     .drop(columns=["_merge"])
```

Product $R \times S$, Theta/Equi/Natural Join

Formal: $R \times S = \{t \circ u \mid t \in R, u \in S\}$; $\bowtie_\theta$ filters product by predicate θ.
ERA (Query): $(R \times S)$ for product; $R \bowtie_{\text{Year}=\text{Year}} S$ for equi-join on `Year`.
Meaning: Combine tuples from both relations; equi-join matches rows sharing the same key(s).
pandas:

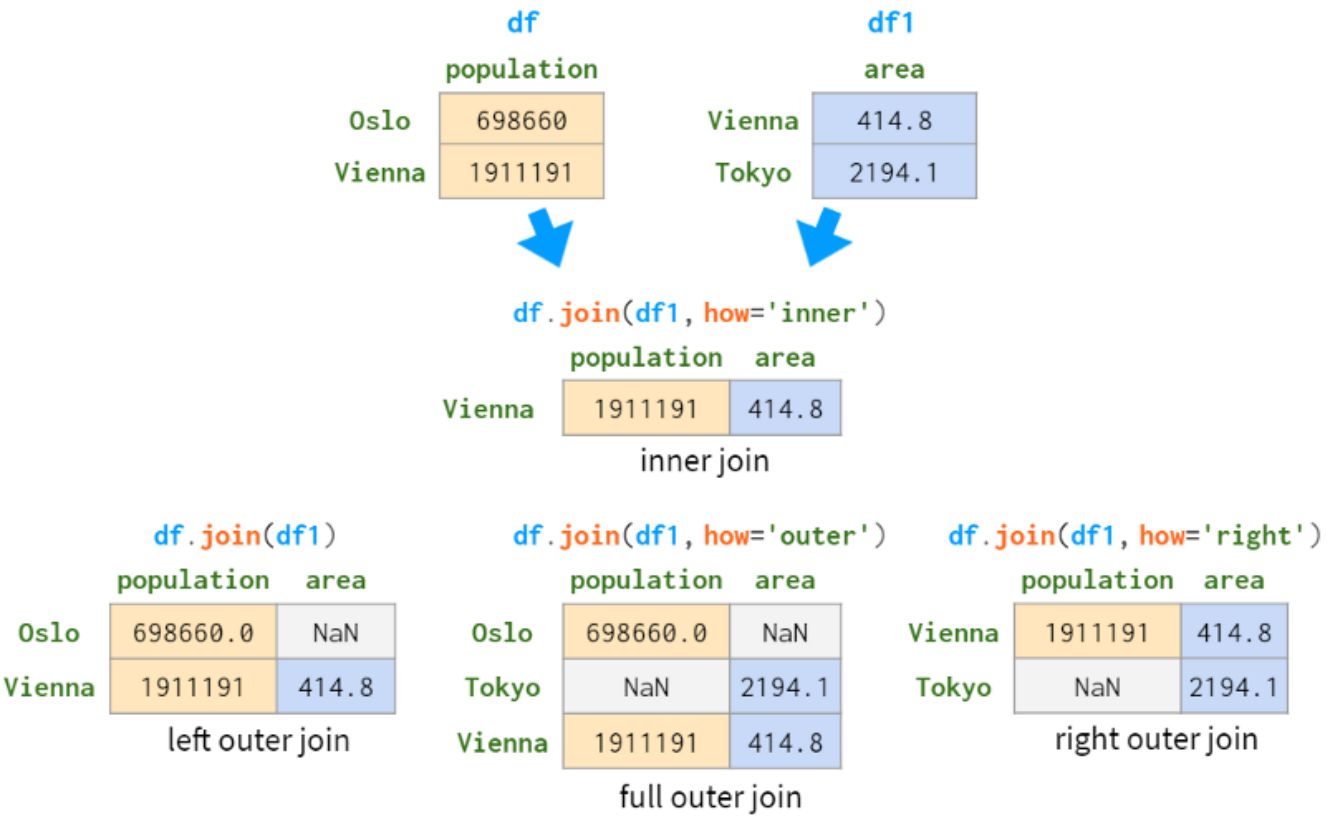
```
cross = pd.merge(R, S, how="cross")  # product

yr_total = (babynames.groupby("Year")["Count"]
            .sum().rename("year_total").reset_index())
eq_join = pd.merge(babynames, yr_total, on="Year", how="inner")
```

Relational Join via `merge` (column keys)



- Basic: `df.merge(df1, on='name')` (default `how='inner'`).
- Other types: `how='left'|'right'|'outer'`.
- Duplicate keys yield a many-to-many (cartesian) match within groups.

DataFrame.join: index-aligned join



- `df.join(df1)` joins on the **index** (set the proper index first or pass `on=` for a left column).
- Join types: `how='left'` (default), `'inner'`, `'outer'`, `'right'`.
- Equivalent to `pd.concat([df, df1], axis=1)` when merely stacking columns without key logic.

Different key names & duplicate columns

cities

	city	state_code
0	San Francisco	CA
1	Miami	FL
2	Washington	DC
3	Los Angeles	CA

states

	code	state
0	CA	California
1	FL	Florida
2	TX	Texas

```
df = cities.merge(states, left_on='state_code', right_on='code')
```

	city	state_code	code	state
0	Los Angeles	CA	CA	Florida
1	San Francisco	CA	CA	California
2	Miami	FL	FL	California

```
df.filter(['city', 'state'])
```

	city	state
0	San Francisco	California
1	Los Angeles	Florida
2	Miami	California

- Resolve column name clashes: `suffixes=('_city','_state')` in `merge`; `lsuffix/rsuffix` in `join`.
- To use an index on the right with `join`:

Outer Joins, Semijoin $R \ltimes S$, Antijoin

Formal: Outer joins keep non-matches with nulls; semijoin returns R tuples with a match in S; antijoin returns those without a match.
ERA (Query): $R \ltimes_{\text{Name}} S, \quad R \setminus (R \ltimes_{\text{Name}} S)$.
Meaning: Keep or exclude R tuples based on existence of a matching Name in S.
pandas:

```
# Outer join (illustrative)
outer = pd.merge(babynames, yr_total, on="Year", how="outer")

# Semijoin (key = Name)
keys = pd.DataFrame({"Name":["Mary","Alice"]})
semi = babynames[babynames["Name"].isin(keys["Name"])]

# Antijoin
anti = (babynames.merge(keys, on="Name", how="left", indicator=
    True)
                .query("_merge=='left_only'")
                .drop(columns=["_merge"]))
```

Grouping/Aggregation $\gamma_{G;\ f_1(A_1)\rightarrow B_1,\ldots}(R)$

Formal: Partition by group attrs G; compute aggregates f_i per group.
ERA (Query): $\gamma_{\text{Sex, decade; mean(Count), max(Count), sum(Count)}}(\text{babynames})$
Meaning: For each `(Sex, decade)` group, compute mean, max, and sum of `Count`.
pandas:

```
babynames["decade"] = (babynames["Year"] // 10) * 10
stats = (babynames.groupby(["Sex","decade"])["Count"]
         .agg(mean="mean", max="max", total="sum")
         .reset_index())
```

Why `reset_index()`?

- **Turn group keys into columns:** By default, `groupby` places `Sex`, `decade` in the *row index* (MultiIndex). `reset_index()` moves them back to regular columns, which is friendlier for downstream tasks.
- **Easier merges/joins:** Many operations expect key columns; with columns, you can `merge(on=["Sex","decade"])` directly (no `right_index=True` needed).

pandas GroupBy as a Mapping: `key` → `subframe/subseries`

Core idea

- `DataFrameGroupBy` represents a **partitioning** of the original DataFrame: a lazy, view-like **mapping** from each **group key value** ⇒ its **sub-DataFrame**.
- `SeriesGroupBy` likewise maps **key** ⇒ **sub-Series**.
- **No copy** is made at grouping time; subframes/series are materialized when accessed (e.g., `for`, `get_group`, `apply/agg`).

Navigation API (view the mapping)

- `for key, subdf in g:` iterate *(key, sub-DataFrame)*
- `g.get_group(key)` ⇒ that group's sub-DataFrame/Series

Examples

```python
import pandas as pd
df = pd.DataFrame({
    "store": ["A","A","B","B","B"],
    "prod":  ["x","y","x","x","y"],
    "qty":   [2, 1, 3, 2, 5],
    "price": [10,12,10,11,12]
})

# DataFrameGroupBy: mapping key -> sub-DataFrame
g = df.groupby(["store","prod"])
first_key, first_subdf = next(iter(g))  # ('A','x'), subframe with those rows
only_Ax = g.get_group(("A","x"))        # same subframe

# SeriesGroupBy: mapping key -> sub-Series
s = df["qty"].groupby(df["store"])
for key, subseries in s:
    _ = (key, subseries.sum())

# Using the mapping to compute results
# (reduce each subframe to a number, then combine)
wmean_price = g.apply(
    lambda sub: (sub["qty"]*sub["price"]).sum() / sub["qty"].sum()
)

# Transform keeps original shape (per-row share within its group)
share_in_store = s.transform(lambda x: x / x.
```

Groupby (DataFrame): Sum and Index Handling

df

client	product	quantity
John	bananas	5
Silvia	oranges	3
Andrew	bananas	4

```
df.groupby('product').sum()
```

product	quantity
bananas	9
oranges	3

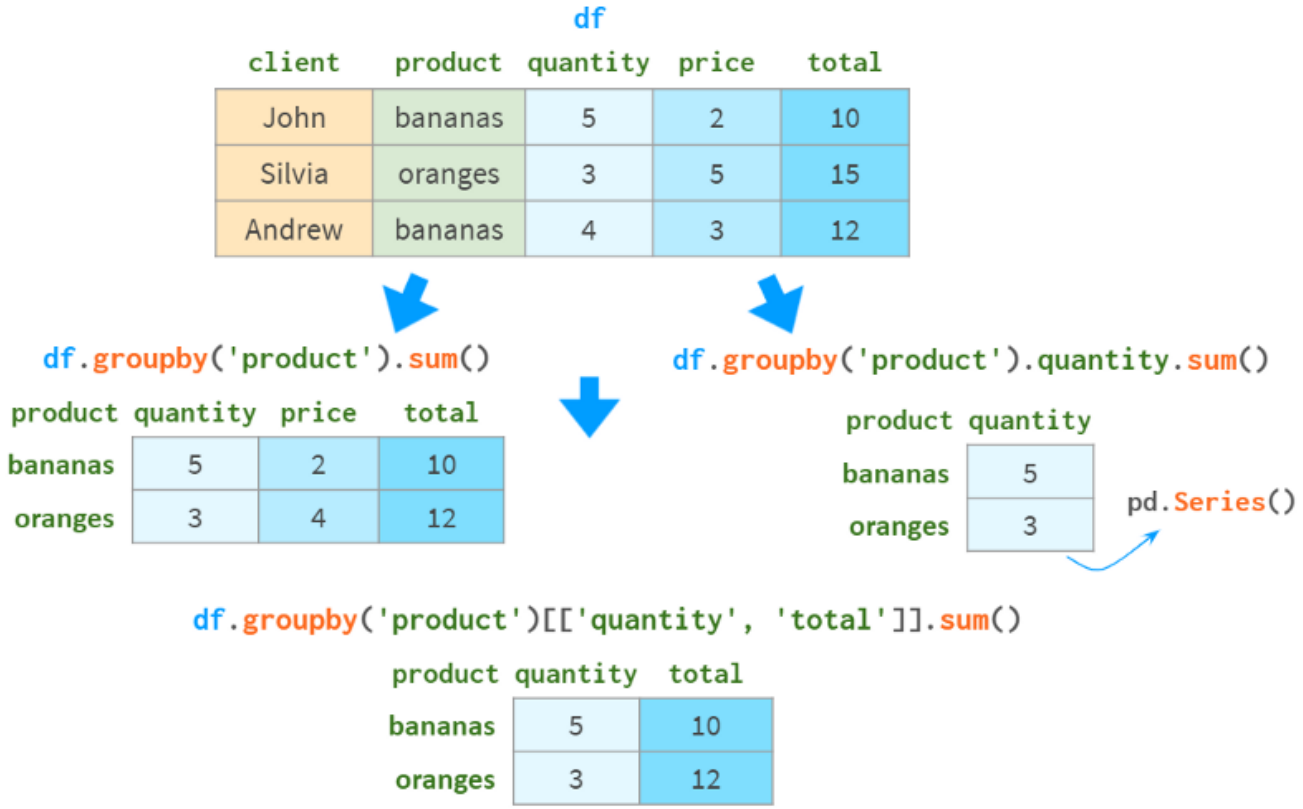
```
.reset_index()
```

```
df.groupby('product', as_index=False).sum()
```

	product	quantity
0	bananas	9
1	oranges	3

- Basic aggregation: `df.groupby('product').sum()`.
- Keep key as a column: `.reset_index()` or use `as_index=False`.
- Only numeric columns are summed; exclude others or select before aggregating as needed.

Groupby: Select Columns vs. Attribute Access



- Aggregate all numeric columns: `df.groupby('product').sum()`.
- Single column Series: `df.groupby('product').quantity.sum()`.
- Subset of columns: `df.groupby('product')[['quantity','total']].sum()` (stays a DataFrame).

Named Aggregations

df

client	product	quantity	price	total
John	bananas	5	2	10
Silvia	oranges	3	5	15
Andrew	bananas	4	3	12

```
df.groupby('product').agg({'quantity': 'sum', 'price': 'mean'})
```

product	quantity	price
bananas	9	2.5
oranges	3	5.

- Assign names while aggregating: `df.groupby('product').agg({'quantity':'sum', 'price':'mean'})`.
- Works with multiple input columns; output columns get meaningful names.

Multiple Aggregations per Column

```
df.groupby('product').agg({'quantity': 'sum', 'price': ['mean', 'max']})
```

	quantity	price	
product	sum	min	max
bananas	9	2	3
oranges	3	5	5

- Multi-agg on a column: `df.groupby('product').agg({'quantity':'sum', 'price':['mean','max']})`.
- Produces a **MultiIndex** on columns; flatten if needed with `map` or list comprehension.

Groupby: Named Aggregations (flat columns)

```
df.groupby('product').agg(quantity=('quantity', 'sum'),
                          min_price=('price', 'min'),
                          max_price=('price', 'max')})
```

product	quantity	min_price	max_price
bananas	9	2	3
oranges	3	5	5

- Use the **named-agg** syntax to avoid MultiIndex columns: `df.groupby('product').agg(quantity=('quantity','sum'), min_price=('price','min'), max_price=('price','max'))`.
- Output columns are exactly the names you provide (`quantity`, `min_price`, `max_price`).

Groupby: Custom Metric via apply

df

client	product	quantity	price	total
John	bananas	5	2	10
Silvia	oranges	3	5	15
Andrew	bananas	4	3	12

```
df.groupby('product').apply(lambda x: x.total.sum()/x.quantity.sum())
```

product	price
bananas	2.44
oranges	5.

- Example weighted mean price per product: `df.groupby('product').apply(lambda x: x.total.sum()/x.quantity.sum())`.
- Returns a Series indexed by group key. Prefer agg if you can express it with built-ins.

Groupby: Mix built-ins with lambdas

df

client	product	quantity	price	total
John	bananas	5	2	10
Silvia	oranges	3	5	15
Andrew	bananas	4	3	12

```
df.set_index('product').groupby('product').agg(total=('total', 'sum'),
fixed_total=('total', lambda x: x.sum()/2 if x.index[0]=='bananas' else x.sum()))
```

product	total	fixed_total
bananas	22	11.
oranges	15	15.

- Combine multiple outputs in one call: `df.groupby('product').agg(total=('total','sum'), fixed_total=('total', lambda x: x.sum()/2 if x.name=='bananas' else x.sum()))`.
- Prefer `x.name` for the group label over inspecting `x.index[0]` (less brittle).

Pivoting I: Wide vs. Long

i \ j	0	1	2
0	a_{00}	a_{01}	a_{02}
1	a_{10}	a_{11}	a_{12}

'short format'

i	j	a
0	0	a_{00}
0	1	a_{01}
0	2	a_{02}
1	0	a_{10}
1	1	a_{11}
1	2	a_{12}

'long format'

- **Wide**: values spread across columns; **Long**: one row per observation with key columns.
- Convert wide $\rightarrow$ long with `stack`/`melt`; long $\rightarrow$ wide with `unstack`/`pivot`.

Pivoting II: `pivot` (long → wide)

df

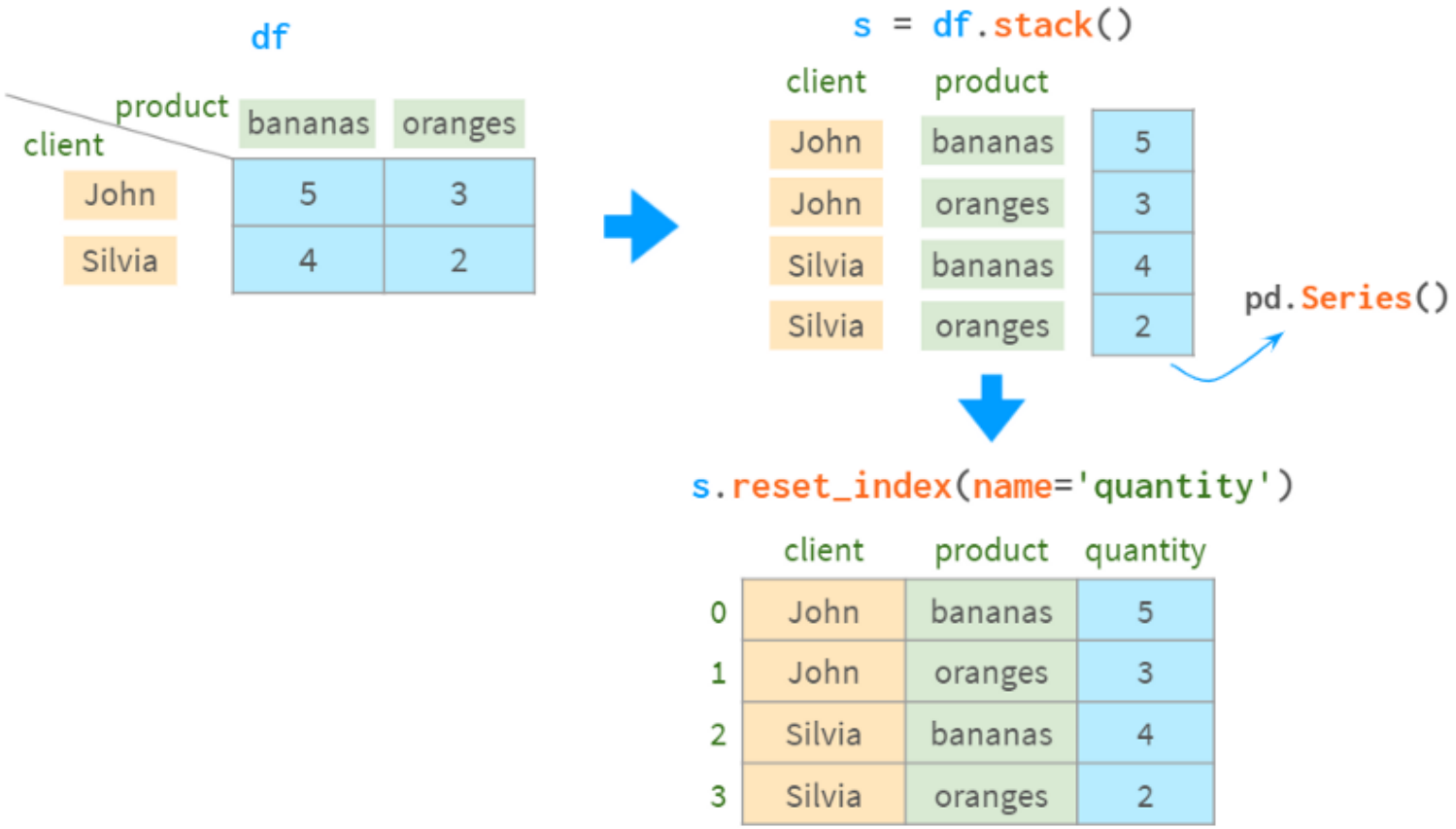
	client	product	quantity	price
0	John	bananas	5	1.5
1	John	oranges	3	3.
2	Silvia	bananas	4	2.5
3	Silvia	oranges	2	4.

```
df.pivot(index='client', columns='product', values='quantity')
```

client \ product	bananas	oranges
John	5	3
Silvia	4	2

- `df.pivot(index='client', columns='product', values='quantity')`.
- Requires a single value per (`index`, `columns`) pair; duplicates raise an error.

Pivoting III: `stack` (wide → long)



- `s = df.stack()` moves column labels into a new index level, returning a Series.
- Recover a tidy DataFrame: `s.reset_index(name='quantity')`.

Pivoting IV: `melt` (wide → long)

df

client \ product	bananas	oranges
John	5	3
Silvia	4	2

```
df1 = df.reset_index()
```

	client	bananas	oranges
0	John	5	3
1	Silvia	4	2

```
df1.melt(id_vars='client', value_vars=['bananas', 'oranges'],
         value_name='quantity')
```

	client	product	quantity
0	John	bananas	5
1	Silvia	bananas	4
2	John	oranges	3
3	Silvia	bananas	4

- `df1 = df.reset_index(); df1.melt(id_vars='client', value_vars=['bananas','oranges'], value_name='quantity')`.
- Choose `melt` when you want explicit control over id/value columns.

Division $R \div S$ ("for all")

Idea: $R(\text{Name}, \text{Year})$; $S(\text{Year})$. $R \div S$ returns the `Names` that appear in *every* `Year` from S.

Toy tables:

R (Name, Year)

Name	Year
Alice	1910
Alice	1911
Bob	1910
Bob	1911
Carol	1910

S (Year)

Year
1910
1911

$R \div S = \{\text{Alice, Bob}\}$

pandas (count-within-S pattern):

```
R = babynames[["Name","Year"]].drop_duplicates()
S = pd.DataFrame({"Year":[1910, 1911]})

need = S["Year"].nunique()
div_result = (R.merge(S, on="Year")
                .groupby("Name")["Year"].nunique()
                .reset_index(name="cnt")
                .query("cnt == @need")["Name"])
```

Why it works: After filtering to years in S, each `Name` must cover all $|S|$ distinct years to satisfy the "for all" condition.

Pivot (Cross-Tab) $\mathrm{PIVOT}_{\text{rows,cols; } f(A)}(R)$

Formal (extended): Reshape R into matrix indexed by rows and cols; each cell is $f(A)$ over that slice.
ERA (Query): $\mathrm{PIVOT}_{\text{rows=Year, cols=Name; } f=\text{count}(*)}\left(\pi_{\text{Year,Name}}(\text{babynames})\right)$
Meaning: A Year $\times$ Name table of occurrence counts; or use $f = \text{sum}(\text{Count})$ for totals.
pandas: `pd.crosstab`, `df.pivot_table`.

```
# Count of names per Year (matrix)
pivot1 = pd.crosstab(index=babynames["Year"], columns=babynames[
    "Name"])

# Sum of Count by (Sex X Year)
pivot2 = babynames.pivot_table(index="Year", columns="Sex",
                               values="Count", aggfunc="sum",
                               fill_value=0, margins=True)
```

Putting It Together (ERA → pandas Pipeline)

ERA (Query):

$$\gamma_{\text{Year,Sex; top1_by(share)}}\ \pi_{\text{Year, Sex, Name, Count, Count/SUM}_y\text{(Count)}\rightarrow\text{share}}\ \sigma_{\text{Year}\geq 1910}(\text{babynames})$$

Meaning: Filter by year, derive per-year share (generalized projection), then pick the top-1 name per `(Year, Sex)` by share.

pandas:

```
result = (babynames
  .query("Year >= 1910")  #
  .assign(year_total=lambda d: d.groupby("Year")["Count"].
      transform("sum"),
          share=lambda d: d["Count"]/d["year_total"])  #   *
  [ ["Year","Sex","Name","Count","share"] ]
  .sort_values(["Year","Sex","share"], ascending=[True,True,
      False])
  .groupby(["Year","Sex"]).head(1)  #    + top-1
  .reset_index(drop=True))
```

Query Best Practices (ERA $\leftrightarrow$ pandas)

- **Selection:** parenthesize masks; consider `.query` for clarity.
- **Generalized projection:** compute with `.assign`, then select explicit columns.
- **Set semantics:** add `.drop_duplicates()` when you want sets, not bags.
- **Joins:** always specify keys; verify key multiplicities to avoid row blow-up.
- **Grouping:** prefer built-ins (`sum`, `mean`, `size`); name outputs.
- **Pivot:** ensure one value per (row,col) or provide an `aggfunc`.

Patterns I — Missing Data

Detect & flag

```
na_rows = df.isna().any(axis=1)
df["has_na"] = na_rows
```

- *What:* Row-level NA indicator for quick filtering, QC, or model exclusions.
- *Why:* Many transforms (groupby, joins) behave differently with NA; keep a provenance flag.
- *Tip:* Use `df.isna().sum()` to profile columns; persist `"has_na"` for audits.

Group-wise impute (median/mean/ffill)

```
df["x_imp"] = (df.groupby("group")["x"]
                 .transform(lambda s: s.fillna(s.median())))
```

- *What:* Impute within homogeneous cohorts to avoid leakage across groups.
- *Choose:* `median` for robustness; `mean` when symmetric; `ffill` for panel data.
- *Gotcha:* Ensure each group has non-missing values; else fallback (e.g., `fillna(global_median)`).

Time-aware fill

```
df = df.sort_values(["id","ts"])
df["y_filled"] = df.groupby("id")["y"].ffill().bfill()
```

- *What:* Forward/backward fill within an entity timeline.
- *Why:* Preserves temporal causality; good for sensors, retail baskets, log streams.
- *Gotcha:* Always sort by entity and timestamp; consider a max gap to avoid stale propagation.

Patterns II — Types & Categories

Coerce numerics safely

```
df["price"] = pd.to_numeric(df["price"], errors="coerce")
```

- *What:* Convert dirty strings (commas, dashes) to numbers; bad parses become `NaN`.
- *Why:* Enables math, winsorization, scaling; surfaces bad entries explicitly.
- *Tip:* Pre-strip currency/commas via `str.replace` before coercion for higher yield.

Use categoricals for keys (memory, speed)

```
df["city"] = df["city"].astype("category")
```

- *What:* Dictionary-encodes repeated labels; reduces memory, speeds groupby/join comparisons.
- *Good for:* Dimensions with low/moderate cardinality (e.g., city, product_id buckets).
- *Gotcha:* Categories are ordered only if set; mismatched category sets across DFs can block merges.

Datetime parsing (robust)

```
df["ts"] = pd.to_datetime(df["ts"], errors="coerce", utc=True)
```

- *What:* Normalize heterogeneous timestamps to timezone-aware UTC.
- *Why:* Makes resampling/rolling reliable; avoids DST bugs.
- *Tip:* When the source TZ is known, first localize (`dt.tz_localize`) then convert to UTC.

Patterns III — Deduplication & Outliers

Exact / subset-based dedup

```
df = df.drop_duplicates(subset=["user_id","ts"])
```

- *What:* Enforce 1 row per key (here, `user_id+ts`).
- *Why:* Prevents double-counting in KPIs and label leakage in ML.
- *Tip:* Decide which duplicate to keep via `sort_values` then call `drop_duplicates(keep="first"/"last")`.

Near-dup by normalized text

```
key = df["name"].str.strip().str.lower()
df = df.loc[~key.duplicated()]
```

- *What:* Collapse cosmetic variants (`" ACME"`, `"acme"`) to a canonical key.
- *When:* Upstream systems differ in case/spacing; basic normalization beats heavier fuzzy matching.
- *Gotcha:* Accents/canonical forms: consider `str.normalize("NFKD")` and `str.replace(r"\s+"," ",regex=True)`.

Winsorize via quantile clipping

```
lo, hi = df["x"].quantile([0.01, 0.99])
df["x_clip"] = df["x"].clip(lo, hi)
```

- *What:* Cap extremes to stabilize means/variances without deleting rows.
- *Why:* Robustifies downstream aggregations and linear models.
- *Tip:* Keep the original alongside (e.g., `x_raw`) for model explainability.

Patterns IV — Reshaping (long ↔ wide)

Wide → Long (tidy)

```
long = df.melt(id_vars=["id"], var_name="k", value_name="v")
```

- *What:* Unpivot columns into key/value rows for tidy analysis, plotting, and `groupby`.
- *Tip:* Add `value_vars=` to melt a specific set of columns only.

Long → Wide (first value per cell)

```
wide = (long.pivot_table(index="id", columns="k",
                         values="v", aggfunc="first")
            .reset_index())
```

- *What:* Pivot back into a matrix; resolve duplicates in a cell via an aggregator.
- *Gotcha:* Duplicates without `aggfunc` will error; pick `first`, `max`, or a custom reducer.

Explode list-like

```
expl = df.explode("tags", ignore_index=True)
```

- *What:* One row per list element (e.g., multi-labels, tokenized text).
- *Tip:* After explode, join with lookup tables to engineer tag-level features.

Patterns V — Join Hygiene & Text/Dates

Join hygiene: validate & audit

```
merged = pd.merge(A, B, on="key", how="left",
                  validate="one_to_one", indicator=True)
anti = merged.loc[merged["_merge"]=="left_only", "key"]
```

- *What:* Enforce expected cardinality and detect missing matches.
- *Why:* Prevents silent row multiplication (*fanout*) and data loss.
- *Tip:* Use `validate="one_to_many"` when you do expect fanout; inspect `"_merge"` distribution.

Text cleanup

```
s = (df["title"].str.normalize("NFKC")
                 .str.strip()
                 .str.lower())
```

- *What:* Canonicalize Unicode, trim whitespace, unify case.
- *Why:* Increases match rates (joins, dedup), stabilizes vectorizers.
- *Tip:* Remove punctuation/#s with `str.replace` (regex) when appropriate.

Datetime gotchas

```
df["ts"] = pd.to_datetime(df["ts"], errors="coerce", utc=True)
```

- *What:* Parse safely and standardize to UTC.
- *Gotcha:* Mixed formats or missing TZ info will coerce to `NaT`; audit with `df["ts"].isna().mean()`.
- *Tip:* After parsing, derive `year`, `week`, `hour` via `.dt` accessors for features.

Patterns VI — Feature Engineering & Pipeline Sanity

Group-aware features: z-score, ranks, rolling

```
g = df.groupby("store")["sales"]
df["z"]    = (df["sales"] - g.transform("mean")) / g.transform("
    std")
df["rank"] = g.rank(ascending=False, method="dense")

df = df.sort_values(["store","ts"])
df["roll7"] = (df.groupby("store")["sales"]
                 .rolling(7, min_periods=1).mean()
                 .reset_index(level=0, drop=True))
```

- *What:* Compute within-group standardization, standings, and moving averages.
- *Why:* Compares entities fairly; captures momentum and seasonality.
- *Tip:* Always sort correctly before rolling; choose `min_periods` to control warm-up bias.

Idempotent chaining with `assign/pipe`

```
tidy = (df
  .pipe(lambda d: d.dropna(subset=["y"]))
  .assign(x=lambda d: d["x"].fillna(d["x"].median()))
  .sort_values(["id","ts"]))
```

- *What:* Declarative pipelines that are re-runnable and testable.
- *Why:* Easier debugging, reproducibility, and unit tests on each stage.
- *Tip:* Use small `pipe` helpers (pure functions) to keep chains readable.